

Manual Técnico

Projeto Garagem — Java / Maven

1. Introdução

Este manual descreve a estrutura técnica do projeto Garagem, desenvolvido em Java com Maven. O projeto utiliza uma organização baseada em MVC, o padrão Strategy, um Repository para acesso à base de dados SQLite e testes unitários com JUnit.

2. Tecnologias utilizadas

- Java 17.
- Apache Maven.
- SQLite através da biblioteca sqlite-jdbc.
- JUnit 5 para testes unitários.
- Maven Surefire para execução dos testes.

3. Estrutura do projeto

```
src/main/java/  
├── Main.java  
├── control/GestorGaragem.java  
├── model/Barco.java  
├── model/Carro.java  
├── model/Garagem.java  
├── model/Matricula.java  
├── model/Mota.java  
├── model/Veiculo.java  
├── repository/GaragemRepository.java  
├── strategy/MovimentoBarco.java  
├── strategy/MovimentoCarro.java  
├── strategy/MovimentoMota.java  
├── strategy/MovimentoStrategy.java  
└── view/GaragemView.java  
src/test/java/  
└── testes unitários  
garagem.db  
pom.xml
```

4. Arquitetura MVC

O projeto separa as responsabilidades em diferentes componentes. O Model representa os dados e regras do domínio; a View apresenta informação ao utilizador; e o Control gere as operações entre os restantes componentes.

- Model: Veiculo, Carro, Mota, Barco, Matricula e Garagem.
- View: GaragemView, responsável pela apresentação da garagem.
- Control: GestorGaragem, responsável por coordenar as operações.
- Repository: GaragemRepository, responsável pelo acesso à base de dados.

5. Principais classes

Veiculo: classe abstrata que contém características comuns aos veículos, como marca, modelo, ano, peso, velocidade máxima e matrícula.

Carro: especialização de Veiculo que possui o número de portas e utiliza MovimentoCarro.

Mota: especialização de Veiculo que possui a cilindrada e utiliza MovimentoMota.

Barco: especialização de Veiculo que possui o tipo de propulsão e utiliza MovimentoBarco.

Matricula: representa os dados da matrícula e distingue matrículas terrestres e marítimas.

Garagem: controla a capacidade e a lista de veículos. A capacidade permitida está entre 0 e 5.

6. Strategy Pattern

O Strategy Pattern é utilizado para separar o comportamento de movimento dos diferentes veículos. A interface MovimentoStrategy define o comportamento, sendo implementada por MovimentoCarro, MovimentoMota e MovimentoBarco.



Cada classe de veículo recebe a estratégia correspondente no seu construtor e o método mover() delega a execução para essa estratégia.

7. Repository e base de dados

A classe `GaragemRepository` é responsável pelo acesso à base de dados SQLite `garagem.db`. A tabela `veiculos` é criada automaticamente caso ainda não exista. O método `buscarTodos()` lê os registos e reconstrói os objetos `Carro`, `Mota` e `Barco`.

Fluxo principal:

*Main → GestorGaragem → GaragemRepository → garagem.db
→ objetos Veiculo → Garagem → GaragemView*

8. Regras da garagem

- A capacidade deve estar entre 0 e 5.
- Não é possível adicionar um veículo nulo.
- Não é possível adicionar veículos quando a garagem está cheia.
- Na sequência inicial, o primeiro veículo deve ser um `Carro`, o segundo um `Barco` e o terceiro uma `Mota`.
- A partir das posições seguintes, a regra de ordem inicial deixa de impedir a entrada.

9. Execução e compilação

*mvn clean compile
mvn exec:java*

Para executar os testes unitários:

mvn test

10. Testes

O projeto contém testes JUnit para as classes `Barco`, `Carro`, `Garagem`, `Matricula`, `Mota` e para o comportamento associado ao Strategy Pattern. Os testes podem ser executados com o comando `mvn test`.

11. Ficheiro pom.xml

O `pom.xml` define o projeto Maven, Java 17, a dependência SQLite, a dependência JUnit 5 e os plugins utilizados para compilação, execução e testes.

12. Fluxo de execução

O Main cria uma Garagem com capacidade 5, cria o GestorGaragem e a GaragemView, carrega os veículos da base de dados, mostra a garagem e, por fim, executa o movimento específico de cada veículo.

13. Manutenção e evolução

Novos tipos de veículos podem ser adicionados através de novas subclasses de Veiculo e de uma nova implementação de MovimentoStrategy. A leitura desses veículos pela base de dados também deverá ser acrescentada ao GaragemRepository.